

Prim's Algorithm

Incremental Construction of Minimum Spanning Trees

1. Introduction to Prim's Algorithm

Prim's algorithm is a greedy algorithm used to find the **Minimum Spanning Tree (MST)** for a weighted undirected graph. It was originally developed by Vojtěch Jarník in 1930 and later independently rediscovered by Robert C. Prim in 1957.

Fundamental Concept: The algorithm grows the MST one vertex at a time, starting from an arbitrary root vertex and always adding the cheapest connection to a vertex outside the current tree.

2. High-Level Logic

The execution of Prim's algorithm can be summarized in four main steps:

1. **Seed:** Select a starting vertex and set its key value to 0. All other vertices start with a key of Infinity.
2. **Select:** Choose the unvisited vertex with the smallest key value (the one closest to the existing MST).
3. **Relax:** For each neighbor of the selected vertex, if the edge weight to that neighbor is less than its current key value, update its key and record the current vertex as its parent.
4. **Iterate:** Repeat until every vertex is included in the MST.

3. Data Structures for Implementation

To implement Prim's efficiently, three primary arrays (or lists) are used:

- **in_mst[]**: Boolean flags to track if a vertex is already part of the tree.
- **key_values[]**: Stores the minimum weight edge connecting a vertex to the MST.
- **parents[]**: Stores the structure of the tree, allowing us to reconstruct the MST once finished.

4. Python Implementation Snippet

```
def prims_algorithm(self):
    key_values = [float('inf')] * self.size
    parents = [-1] * self.size
    in_mst = [False] * self.size
    key_values[0] = 0 # Start at vertex 0

    for _ in range(self.size):
        # Find min key vertex not in MST
        u = min((v for v in range(self.size) if not in_mst[v]),
                key=lambda v: key_values[v])
        in_mst[u] = True

        # Update neighbors
        for v in range(self.size):
            weight = self.adj_matrix[u][v]
            if 0 < weight < key_values[v] and not in_mst[v]:
                key_values[v] = weight
                parents[v] = u
```

5. Time Complexity Analysis

The performance of Prim's algorithm depends on the graph's representation:

- **Adjacency Matrix + Linear Search:** $O(V^2)$. This is efficient for dense graphs.
- **Adjacency List + Binary Heap:** $O(E \log V)$. This is preferred for sparse graphs where edges are fewer.
- **Fibonacci Heap:** $O(E + V \log V)$. The state-of-the-art for extremely large networks.

CodeHive

Comparison: Prim vs. Dijkstra (Part 1)

It is common to confuse Prim's and Dijkstra's because they both use key values and relaxation. However, the **Relaxation Goal** is different:

Dijkstra: Minimize distance from the *source* to all other vertices. (key = $\text{dist}[u] + \text{weight}$)

Prim: Minimize distance from the *entire MST* to the next vertex. (key = weight)

Sparse vs. Dense Graphs

In a sparse graph (where $E \approx V$), the edge updates are infrequent, and using a Priority Queue significantly speeds up the selection of the next vertex. In a dense graph (where $E \approx V^2$), the nested loops of the matrix-based implementation are quite effective because the overhead of managing a heap may outweigh the benefits of log-time selection.

Comparison: Prim vs. Dijkstra (Part 2)

It is common to confuse Prim's and Dijkstra's because they both use key values and relaxation. However, the **Relaxation Goal** is different:

Dijkstra: Minimize distance from the *source* to all other vertices. (key = $\text{dist}[u] + \text{weight}$)

Prim: Minimize distance from the *entire MST* to the next vertex. (key = weight)

Sparse vs. Dense Graphs

In a sparse graph (where $E \approx V$), the edge updates are infrequent, and using a Priority Queue significantly speeds up the selection of the next vertex. In a dense graph (where $E \approx V^2$), the nested loops of the matrix-based implementation are quite effective because the overhead of managing a heap may outweigh the benefits of log-time selection.